

"SMOTEEN based Boosting method: Optimal Sampling Strategies for Imbalanced Software Defect Classification"

Abstract

In the field of software defect prediction, class imbalance is a common problem. Database imbalances have a substantial effect on defect prediction models' accuracy. Various solutions have been suggested to mitigate this issue, such as algorithmic approaches, oversampling, and under sampling. The SMOTEEN+LGBM method, which combines the SMOTEEN resampling methodology with the Light Gradient Boosting Machine (LGBM) classifier, is a novel solution that is presented in this paper. Especially, when it comes to managing unbalanced datasets, this solution performs better than conventional ones. The effectiveness of the suggested approach highlights the necessity for extensive validation using various software defect datasets and statistical testing to determine the significance of noted improvements. Although designed for coursework, this work lays the groundwork for future investigations by emphasizing the need for thorough comparisons with cutting-edge techniques in the hunt for the best ways to handle class imbalance in software defect prediction.

1. Introduction

A crucial component of data mining and knowledge discovery in databases (KDD) is classification[1]. Learning a function from training data is the goal of classification modeling, which produces as little inaccuracies as much as feasible when applied to previously unseen data. Several algorithms for modeling classification, including well-developed and effectively used algorithms include decision trees, neural networks, Bayesian networks, nearest neighbors, support vector machines, and the recently published associative classification utilized in a variety of application areas[2]. Nevertheless, reports from research and business alike show that uneven class distribution in a dataset has presented significant challenges for the majority of techniques for classifier learning, which presuppose a rather balanced distribution. It is said that there are significantly more instances of some classes than others in the unbalanced class distribution.

Because standard classifiers are built to generalize from training data and produce the simplest hypothesis that best matches the data, they typically perform poorly on unbalanced data sets. In contrast, the simplest hypothesis overlooks infrequent instances in an unbalanced set of data. Categorization guidelines that forecast the tiny class are therefore typically weaker and fewer in number than those that indicate the prevalent class. As a result, test samples from the little are mislabeled more frequently than those who belong to the common class. In the majority of cases, even when the level of unbalance differs depending on the application, the appropriate samples in the unusual class are frequently classified with a higher greater than the alternative scenario[3]. So, the imbalanced dataset can be the effects of different class imbalanced problems as mentioned by imbalanced class distribution,

small sample size, separability, within class imbalanced[3].

The reported research solution for class imbalanced problem can be distinguished into data level approaches and algorithm level approaches. The goal is to resample the data space in order to rebalance the class distribution at the data level. In order to improve learning in relation to the tiny class, algorithms are tried to be adjusted at the algorithmic level. Cost-sensitive learning solutions aim to reduce high-cost errors by assuming greater misclassification costs for samples in the rare class and combining both algorithmic level approaches and data[2]. In addition, a number of boosting algorithms are described as meta-techniques to address the issue of class imbalance.

2. Related Work

The term "software defect prediction" (SDP) describes methods that establish a connection between software metrics and software defects using previous defect data. SDP has made use of a wide range of software metrics, such as method-level metrics (such as the widely used McCabe and Halstead metrics), Chidamber and Kemerer's (CK) metrics, which are specifically designed for object-oriented software, network metrics, cascading style sheet metrics, web metrics, change metrics, developer micro interaction metrics, mutation-based metrics, package modularization metrics, context-based package cohesion metrics, lines of comments, and code smell metrics[4].

Machine learning techniques such as artificial neural networks Bayesian networks, support vector machine, dictionary learning, association rule, naive Bayes, tree-based methods, evolutionary algorithm, and ensemble learning are the most well-liked and frequently used modeling approaches for SDP. Certain studies developed fuzzy inference-based SDP models

because they believed that software defect-proneness has the fuzzy characteristic. There are situations when there are very few or no labeled examples available for training SDP models. As a result, some researchers have developed semi-supervised and unsupervised approaches for SDP[1], [4], [5].

To handle the problem of data imbalance case in software defect, data based and algorithm centric approach were proposed. In the context of data level approach, under sampling, over sampling as well as the hybrid approach were proposed to handle the class imbalance problem[1]–[10]. In addition to this, algorithm-based approach such as weighted random forest, cost sensitive learning, bootstrapping strategy were proposed to handle class imbalance problems[4], [5], [11]. However, the effect of the oversampling, hybrid method with boosting approach had not been explored, to understand how well this methodology can solve the problem of data imbalance case[12].

3. Methodology

3.1 Experimental Setup

3.1.1 Dataset

JM1 Dataset

Imbalance dataset called as JM1 was selected for the experimental purpose to evaluate the effectiveness of the proposed algorithm[4]. Originally, this dataset consists of 112648 samples with 23 feature variables as shown in Figure 1. Then, the condition was applied such that $loc > 300$, $l > 1.2$, $e > 300000000$, $b > 25$, $t > 1500000$, $lOCCode > 2500$, $locCodeandComment > 100$, $total_Op > 5000$, $total_Opnd > 3000$ were treated

as outliers to improve the model performance. So, 112639 samples were retrieved for the experimental purpose. The final dataset consists of 25166 positive samples and 87473 negative samples with 21 features attributes as shown in Figure 2 with imbalance distributions.

Data columns (total 23 columns):				
#	Column	Non-Null	Count	Dtype
0	id	101763	non-null	float64
1	loc	112648	non-null	float64
2	v(g)	112648	non-null	float64
3	ev(g)	112648	non-null	float64
4	iv(g)	112648	non-null	float64
5	n	112648	non-null	float64
6	v	112648	non-null	float64
7	l	112648	non-null	float64
8	d	112648	non-null	float64
9	i	112648	non-null	float64
10	e	112648	non-null	float64
11	b	112648	non-null	float64
12	t	112648	non-null	float64
13	lOCCode	112648	non-null	int64
14	lOCComment	112648	non-null	int64
15	lOBlank	112648	non-null	int64
16	locCodeAndComment	112648	non-null	int64
17	uniq_Op	112648	non-null	object
18	uniq_Opnd	112648	non-null	object
19	total_Op	112648	non-null	object
20	total_Opnd	112648	non-null	object
21	branchCount	112648	non-null	object
22	defects	112648	non-null	bool

dtypes: bool(1), float64(13), int64(4), object(5)
memory usage: 19.9+ MB

Figure 1: Attributes for the software defect prediction.

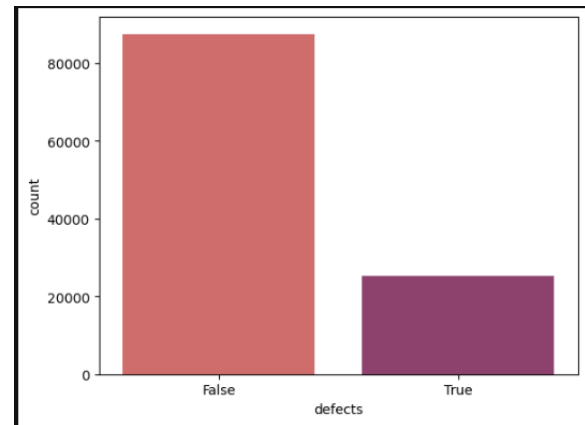


Figure 2: Bar plot to represent class distribution used for software defect prediction(JM1 DATASET)..

3.2 Classifiers

3.2.1 LGBM Classifier

Microsoft created the potent open-source machine learning system known as LightGBM, or Light Gradient Boosting Machine[13]. It belongs to the class of gradient boosting frameworks and is made with efficiency and performance in mind. LightGBM uses a leaf-wise technique instead of the conventional depth-first tree development strategy, which helps to speed up training times. During tree construction, the framework computes information gain using histogram-based techniques, which improves speed and memory economy, especially for large datasets. Notably, LightGBM does not require a lot of one-hot encoding because it provides native support for categorical characteristics. It is also very scalable, supporting both distributed and parallel training, which makes it ideal for distributed computing settings and huge datasets[13]. The framework is renowned for using GPU resources effectively, which speeds up prediction and training.

Light Gradient Boosting Machine, or LightGBM, is an excellent classifier of imbalanced datasets because of its sophisticated features and optimization methods. By utilizing the gradient boosting framework, LightGBM creates a group of weak learners—mostly decision trees—that are skilled at identifying patterns in unbalanced data sets. The ability of the model to include a weighted objective function is essential because it permits differentiating weighting for different groups and placing more focus on the minority class, which corrects the imbalance[1]. Subsampling is incorporated into LightGBM during training to further improve its efficacy. This ensures that the minority class is represented in each subset and keeps the dominant class from overshadowing it. The implementation of the leaf-wise tree growth approach makes it easier to capture complex

patterns in the minority class, which is a significant benefit in unbalanced settings. The model can handle big and imbalanced datasets thanks to its native support for categorical features, effective feature parallelism, and memory optimization. The ensemble learning paradigm of LightGBM combines predictions from several weak learners to produce improved overall predictive performance, which is especially useful in the presence of unbalanced data distributions. LightGBM is an extremely useful tool for accurately and efficiently categorizing imbalanced datasets due to the convergence of these technological features.

3.2.2 SMOTENN+LGBM Classifier

Using edited closest neighbors to clean up the synthetic data and oversampling the minority class, SMOTENN (Synthetic Minority Over-sampling strategy coupled with Edited closest Neighbors) is a strategy for handling imbalanced datasets[14], [15]. The procedure entails training the model on the resampled dataset and combining it with an LGBM (Light Gradient Boosting Machine) classifier. The following steps are included in the mathematical equation for SMOTENN+LGBM classification and represented

as

:

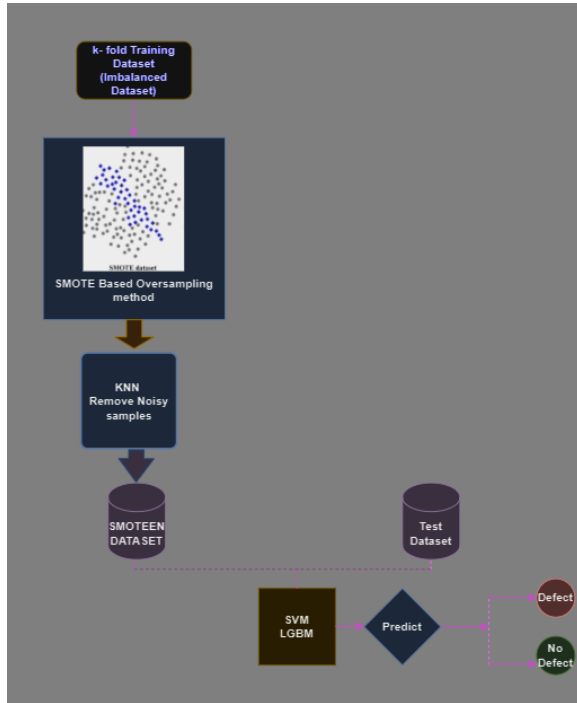


Figure 3: Data flow diagram for SMOTENN based ML architecture.

- i. **SMOTENN Resampling**
 $X_{\text{resampled}}, Y_{\text{resampled}} = \text{SMOTENN}(X_{\text{original}}, Y_{\text{original}})$
- ii. **LGBM Training**
 $\text{LGBM.fit}(X_{\text{resampled}}, Y_{\text{resampled}})$
- iii. **Prediction**
 $\hat{Y} = \text{LGBM.predict}(X_{\text{test}})$

This proposed methodology supports the LGBM model better handle imbalanced dataset by providing more balanced training set, improving its ability to learn from minority class.

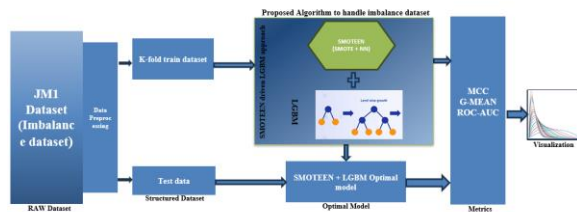


Figure 4: Proposed framework for handling imbalanced dataset in software defect prediction.

The proposed architecture for data imbalance problem is as shown in Figure 4 and Figure 4:

Proposed framework for handling imbalanced dataset in software defect prediction.. The JM1 dataset were transform, outliers were removed and transformed using a standardization so, that each attributes have equal contribution for prediction of software defect. Then the dataset is divided into a train dataset for training, and test dataset for evaluation of proposed method. The 4-fold evaluation strategy was used to evaluate the performance of the training dataset using the performance metrics frequently used in imbalance case such as MCC, G-mean and ROC-AUC [16]. The proposed methodology called as SMOTENN+LGBM model is used to evaluate the performance on test dataset, selecting the metrics specific to imbalance case measure where ROC-AUC has been visualized to ensure that model have not suffer a problem of underfitting and overfitting.

The pseudo code for the proposed method is represented below as:

Pseudo-code for SMOTE-EN + LGBM for Software Defect Prediction

Step 1: Import necessary libraries

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from imblearn.combine import SMOTENN
from lightgbm import LGBMClassifier

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
```

Step 2: Load the dataset (replace 'your_dataset.csv' with the actual dataset file)

```
data = pd.read_csv('your_dataset.csv')
```

Step 3: Preprocess the data (handle missing values, encode categorical variables, etc.)

Step 4: Split the data into features (X) and target variable (y)

```
X = data.drop('defect_label', axis=1) # Assuming 'defect_label' is
the target variable
```

```
y = data['defect_label']
```

Step 5: Split the data into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Step 6: Apply SMOTE-EN for oversampling

```
smote_en = SMOTENN(random_state=42)
```

```
X_resampled, y_resampled = smote_en.fit_resample(X_train,
y_train)
```

Step 7: Train the LightGBM model

```
lgbm_model = LGBMClassifier(random_state=42)
```

```
lgbm_model.fit(X_resampled, y_resampled)
```

Step 8: Make predictions on the test set

```
y_pred = lgbm_model.predict(X_test)
```

Step 9: Evaluate the model performance

```
accuracy = accuracy_score(y_test, y_pred)
```

```
conf_matrix = confusion_matrix(y_test, y_pred)
```

```
classification_report = classification_report(y_test, y_pred)
```

Step 10: Display results

```
print(f"Accuracy: {accuracy}")
```

```
print("Confusion Matrix:")
```

```
print(conf_matrix)
```

```
print("Classification Report:")
```

```
print(classification_report)
```

3.3 Evaluation Metrics

Mathew's correlation coefficient

Mathew's correlation coefficient abbreviated as MCC and a popular measure metrics in classification problems. MCC can be mathematically represented as:

$$MCC = (TP \cdot TN - FP \cdot FN) / \sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}$$

ROC Curve

Mathematically, ROC values measure the performance of the classifiers at different threshold with respect to TPR versus FPR. A higher value ROC indicate that the model has better performance at classifying positive and negative instance for the imbalance case also. For example, AUC-ROC = 0.5 indicates that the classifier performs no better than random

chance and ROC-AUC > 0.5 indicate the classifier performs better than random chance. Similarly, ROC-AUC = 1, signifies that the model achieves perfect discrimination between the classes.

G-Mean

G-mean provides the balance measure of the classifier's performance particular in situation where there is an imbalance between the classes. Mathematically, G-mean is calculated using the following formula:

$$G\text{-mean} = \sqrt{(\text{Sensitivity} * \text{Specificity})}$$

Here, Sensitivity is the true positive rate (TPR) and specificity is the true negative rate (TNR). G-mean is the geometric mean of these two values.

3.4 Research Questions

In order to interpret the performance of the proposed strategy for the class imbalance problem, we set up three research question to explain the performance of the model:

- **RQ1:** How effective is **LGBM-SMOTENN** to the **SVM (base classifier) with no oversampling**, no oversampling but **boosting** and **SMOTEEN with SVM**?
- **RQ2:** How effective is our proposed **LGBM-SMOTENN** to **Cost Sensitive learning**?
- **RQ3:** How effective is our proposed **LGBM-SMOTENN** to the **over-sampling method (SMOTE) based boosting method** and **SMOTE based SVM model**?

4. Experiments and Research Results

RQ1: How effective is **LGBM-SMOTENN** to the **SVM (base classifier) with no oversampling**, no oversampling but **boosting** and **SMOTEEN with SVM**?

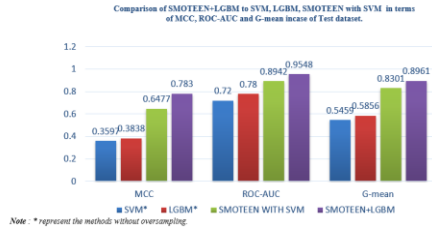


Figure 5: Comparison of SMOTEEN+LGBM method to base classifier, boosting, SMOTEEN with SVM.

In order to validate the performance of model, we compare the result with respect to SVM model as the base classifier, LGBM(Boosting) and SMOTEEN with SVM and SMOTEEN+LGBM method. The metrics for evaluation were MCC, ROC-AUC and G-mean. Comparatively, the boosting method perform better than the linear model for the classification. However, the combined strategy called as SMOTENN+LGBM outperforms the other strategy in terms of the metrics. SVM method achieved MCC, ROC-AUC and G-mean 0.3597, 0.72 and 0.5459. Similarly, LGBM method achieved MCC, ROC-AUC and G-mean of 0.3838, 0.78, 0.5856 which shows that boosting method performs well than the base classifier. SMOTEEN with SVM perform better than base classifier and LGBM classifier, which shows the importance of SMOTEEN method for imbalance case. Comparatively, SMOTEEN+LGBM method highest value in terms of MCC, ROC-AUC, G-mean which signify that the proposed strategy can perform outstanding in case of imbalance dataset too as shown in Figure 5.

RQ2: How effective is our proposed **LGBM-SMOTEEN** to **LGBM-CS**?

In order to verify that the proposed strategy is superior to the algorithmic approach, we compare the

performance of the LGBM-SMOTEEN method with LGBM-CS method as shown in Figure 6.

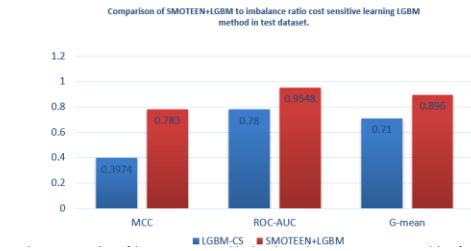


Figure 6: Comparison of the LGBM-Cost Sensitive learning to the SMOTEEN-LGBM model performance for SDP imbalance dataset.

The experimental result verifies that, SMOTEEN+LGBM method achieves better performance than LGBM-cost sensitive method incases of imbalance case. LGBM-CS achieved MCC, ROC-AUC and G-mean of 0.3974, 0.78, 0.71 and SMOTEEN+LGBM achieved MCC, ROC-AUC, G-mean of 0.783, 0.9548, 0.890 which is relatively higher than LGBM-CS as shown in Figure 6. This experimental result verifies that, the proposed strategy performs well than the cost-sensitive based learning in case of imbalance dataset.

RQ3: How effective is our proposed **LGBM-SMOTEEN** to the Oversampling *only* (**SMOTE based boosting method** and **SMOTE based SVM model**)?

In order to verify the effectiveness of the proposed strategy, we compare the performance of the proposed strategy with oversampling method in case of base classifier and boosting method too.

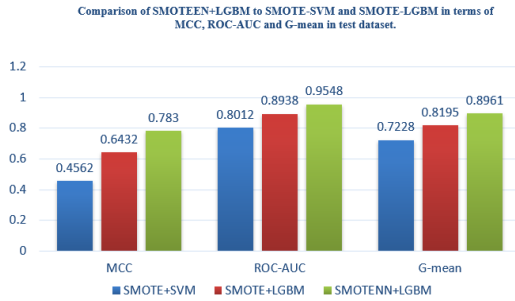


Figure 7: Comparison of the SMOTE+SVM, SMOTE+LGBM and SMOTEEN+LGBM for imbalance dataset in SDP.

The experimental result verifies that, SMOTEEN+LGBM method achieved optimal performance with respect to SMOTE+SVM, SMOTE+LGBM in terms of MCC, ROC-AUC and G-mean as shown in Figure 7.

5. Conclusion and Future Works

To sum up, class imbalance in the software defect domain still has to be addressed because it can seriously affect how well defect prediction models work. A number of tactics have been put out to address this problem, including oversampling, under sampling, and algorithmic techniques like cost-sensitive learning and bootstrapping. On the other hand, the suggested algorithm, known as SMOTEEN+LGBM, has proven to perform better than earlier approaches.

A balanced and precise categorization of software flaws has been achieved through the use of the Light Gradient Boosting Machine (LGBM) classifier in conjunction with the SMOTEEN resampling technique. By utilizing the advantages of both oversampling and cleaning through edited nearest neighbors, this strategy performs better than conventional ones. The suggested strategy's overall success can be attributed to the robustness and efficiency of the LGBM classifier.

It is imperative to recognize the limitations of this work, nevertheless. It is crucial to conduct validation using various software defect datasets in order to guarantee the applicability of the suggested SMOTEEN+LGBM approach. Statistical tests are also necessary to validate the reliability of the data and determine the significance of the observed improvements.

It is also critical to acknowledge that the suggested approach is designed for academic coursework, and that comparing it to other approaches offers an interesting direction for further investigation. Undertaking thorough comparison evaluations using alternative tactics put forward will enhance comprehension of the advantages and drawbacks of different methods concerning software defect prediction.

In conclusion, even though SMOTEEN+LGBM shows promise in addressing class imbalance in software defect prediction, further study and thorough testing on a variety of datasets are required for its validation and improvement. The foundation for further research into the best approaches to correcting class imbalance in the field of software defects is laid by this work.

References

- [1] S. Feng, J. Keung, X. Yu, Y. Xiao, and M. Zhang, "Investigation on the stability of SMOTE-based oversampling techniques in software defect prediction," *Inf Softw Technol*, vol. 139, Nov. 2021, doi: 10.1016/j.infsof.2021.106662.
- [2] Q. Song, Y. Guo, and M. Shepperd, "A Comprehensive Investigation of the Role of Imbalanced Learning for Software Defect Prediction," *IEEE Transactions on Software Engineering*, vol. 45, no. 12, pp. 1253–1269, Dec. 2019, doi: 10.1109/TSE.2018.2836442.
- [3] H. Ali, M. N. M. Salleh, R. Saedudin, K. Hussain, and M. F. Mushtaq, "Imbalance class problems in data mining: A review," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 14, no. 3, pp. 1552–1563, Jun. 2019, doi: 10.11591/ijeecs.v14.i3.pp1552-1563.
- [4] H. Tong, B. Liu, and S. Wang, "Software defect prediction using stacked denoising autoencoders and two-stage ensemble learning," *Inf Softw Technol*, vol. 96, pp. 94–111, Apr. 2018, doi: 10.1016/j.infsof.2017.11.008.
- [5] Y. Sun, M. S. Kamel, A. K. C. Wong, and Y. Wang, "Cost-sensitive boosting for classification of imbalanced data," *Pattern Recognit*, vol. 40, no. 12, pp. 3358–3378, Dec. 2007, doi: 10.1016/j.patcog.2007.04.009.
- [6] Z. Li, X. Y. Jing, and X. Zhu, "Progress on approaches to software defect prediction," *IET Software*, vol. 12, no. 3. Institution of Engineering and Technology, pp. 161–175, Jun. 01, 2018. doi: 10.1049/iet-sen.2017.0148.
- [7] D. Rodriguez, I. Herraiz, R. Harrison, J. Dolado, and J. C. Riquelme, "Preliminary comparison of techniques for dealing with imbalance in software defect prediction," in *ACM International Conference Proceeding Series*, Association for Computing Machinery, 2014. doi: 10.1145/2601248.2601294.
- [8] L. Qiao, X. Li, Q. Umer, and P. Guo, "Deep learning-based software defect prediction," *Neurocomputing*, vol. 385, pp. 100–110, Apr. 2020, doi: 10.1016/j.neucom.2019.11.067.
- [9] I. H. Laradji, M. Alshayeb, and L. Ghouti, "Software defect prediction using ensemble learning on selected features," *Inf Softw Technol*, vol. 58, pp. 388–402, Feb. 2015, doi: 10.1016/j.infsof.2014.07.005.
- [10] R. Malhotra and S. Kamal, "An empirical study to investigate oversampling methods for improving software defect prediction using imbalanced data," *Neurocomputing*, vol. 343, pp. 120–140, May 2019, doi: 10.1016/j.neucom.2018.04.090.
- [11] S. Feng, J. Keung, X. Yu, Y. Xiao, and M. Zhang, "Investigation on the stability of SMOTE-based oversampling techniques in software defect prediction," *Inf Softw Technol*, vol. 139, Nov. 2021, doi: 10.1016/j.infsof.2021.106662.
- [12] B. W. Boehm, "Software Engineering Economics," 1984.
- [13] J. Fan, X. Ma, L. Wu, F. Zhang, X. Yu, and W. Zeng, "Light Gradient Boosting Machine: An efficient soft computing model for estimating daily reference evapotranspiration with local and external meteorological data," *Agric*

Water Manag, vol. 225, Nov. 2019, doi:
10.1016/j.agwat.2019.105758.

- [14] F. Matloob *et al.*, “Software defect prediction using ensemble learning: A systematic literature review,” *IEEE Access*, vol. 9. Institute of Electrical and Electronics Engineers Inc., pp. 98754–98771, 2021. doi: 10.1109/ACCESS.2021.3095559.
- [15] S. Maldonado, J. López, and C. Vairetti, “An alternative SMOTE oversampling strategy for high-dimensional datasets,” *Applied Soft Computing Journal*, vol. 76, pp. 380–389, Mar. 2019, doi: 10.1016/j.asoc.2018.12.024.
- [16] Q. Zou, S. Xie, Z. Lin, M. Wu, and Y. Ju, “Finding the Best Classification Threshold in Imbalanced Classification,” *Big Data Research*, vol. 5, pp. 2–8, Sep. 2016, doi: 10.1016/j.bdr.2015.12.001.